CSM GLEAMING 2

COMPUTER SCIENCE MENTORS 61A

November 9, 2026 – November 13, 2026

1 More Gleam!

These questions are adopted from past CS 61A exams related to linked lists, trees, and/or Scheme.

Here is some commonly used list syntax:

- `[]` is empty.
- `[first, ..rest]` splits a non-empty list.
- `[first, ..rest]` also constructs a new list.
- `#(x, y)` is a two-element tuple.

Here are some commonly used built-in list functions:

- **`list`**`.append(xs, ys)` joins two lists.
- **`list.map`**`(xs, f)` applies `f` to each value.
- **`list.filter`**`(xs, keep)` keeps values satisfying `keep`.
- **`list`**`.length(xs)` returns a list's length.

You can find other useful builtin functions in Gleam here:
https://gleam-stdlib.hexdocs.pm/gleam/list.html
https://gleam-stdlib.hexdocs.pm/gleam/int.html
https://gleam-stdlib.hexdocs.pm/gleam/result.html

Created by Ana Abell, Terry Hsu. feedback form!
Provided by CSM 61A contributors under CC BY 4.0/3-clause BSD.

1. *Adapted from CS 61A Fall 2024 Final Q7 (A Pair of Schemes)*

   (a) Implement `all_pairs`, which takes a two-argument function `f` and a list `s`. It returns `True` if `f(x, y)` returns `True` for every pair of adjacent elements `x` and `y` in `s`. The function `inc` returns whether `x + 1` equals `y`.

```
pub fn inc(x: Int, y: Int) -> Bool {
  x + 1 == y
}

pub fn all_pairs(f: fn(a, a) -> Bool, s: List(a)) -> Bool {
  case s {
    [x, y, ..rest] -> _____________ && all_pairs(f, _____________)
    _ -> True
  }
}
assert all_pairs(inc, [3, 4, 5, 6, 7, 8]) == True
assert all_pairs(inc, [3, 4, 5, 8, 7, 8]) == False
assert all_pairs(inc, [3, 4, 6, 7]) == False
assert all_pairs(inc, [3]) == True
assert all_pairs(inc, []) == True
```

   (b) Implement `show_pairs`, which takes a list `s` and returns a list of all pairs of adjacent elements in `s`, in order. Each pair is a tuple.

```
pub fn show_pairs(s: List(a)) -> List(#(a, a)) {
  case s {
    [x, y, ..rest] -> [__________, ..show_pairs(_____________)]
    _ -> []
  }
}
assert show_pairs([3, 5, 7, 9, 11, 13])
  == [#(3, 5), #(5, 7), #(7, 9), #(9, 11), #(11, 13)]
assert show_pairs([1, 2]) == [#(1, 2)]
assert show_pairs([1]) == []
```

   (c) Implement `all_pairs_calls`, which takes a two-argument function `f` and a list `s`. It returns one *delayed call* (a function that takes no arguments) for each pair of adjacent elements `x` and `y` in `s`, which calls `f(x, y)` when it is called. Then implement `run_all`, which calls each delayed call in a list and returns whether all of them returned `True`. You may use `show_pairs`.

```
pub fn all_pairs_calls(f: fn(a, a) -> Bool, s: List(a)) -> List(fn()
   -> Bool) {
  list.map(show_pairs(s), fn(pair) { ___________________________ })
}

pub fn run_all(calls: List(fn() -> Bool)) -> Bool {
  list.all(calls, _____________________)
}
assert list.length(all_pairs_calls(inc, [3, 4, 5, 6, 7, 8])) == 5
assert run_all(all_pairs_calls(inc, [3, 4, 5, 6, 7, 8])) == True
assert run_all(all_pairs_calls(inc, [3, 4, 5, 8, 7, 8])) == False
assert run_all([]) == True
```

2. *Adapted from CS 61A Fall 2022 Final Q4b (Advent of Code)*

   Implement `find_pair`, which takes a list of integers `s` and returns the two elements of `s` that sum to 2020, in the order that they appear in `s`. Assume that exactly one pair of elements sums to 2020. You may call **`list.`**`contains(s, x)`, which returns whether `s` contains `x`.

```
pub fn find_pair(s: List(Int)) -> List(Int) {
  case s {
    [] -> []
    [x, ..rest] ->
      case ____________________________________ {
        True -> ___________________
        False -> ___________________
      }
  }
}
assert find_pair([979, 1721, 366, 299, 675, 1456]) == [1721, 299]
assert find_pair([2000, 20]) == [2000, 20]
assert find_pair([1, 1010, 3, 1010]) == [1010, 1010]
```

3. *Adapted from CS 61A Fall 2014 Midterm 2 Q3b (This One Goes to Eleven)*

   Implement `no_eleven`, which takes a non-negative integer `n`. It returns a list of all distinct lists of length `n` that contain only `1`s and `6`s, in which two `1`s never appear next to each other.

```
pub fn no_eleven(n: Int) -> List(List(Int)) {
  case n {
    0 -> _________
    1 -> [[6], [1]]
    _ ->
      list.append(
        list.map(_______________, fn(s) { [6, ..s] }),
        list.map(_______________, fn(s) { ________________ }),
      )
  }
}
assert no_eleven(0) == [[]]
assert no_eleven(2) == [[6, 6], [6, 1], [1, 6]]
assert no_eleven(3)
  == [[6, 6, 6], [6, 6, 1], [6, 1, 6], [1, 6, 6], [1, 6, 1]]
assert no_eleven(4)
  == [[6, 6, 6, 6], [6, 6, 6, 1], [6, 6, 1, 6], [6, 1, 6, 6],
      [6, 1, 6, 1], [1, 6, 6, 6], [1, 6, 6, 1], [1, 6, 1, 6]]
```

4. *Adapted from CS 61A Spring 2018 Midterm 2 Q4b (Sequences)*

   Implement `combo`, which takes two lists of digits `a` and `b`. It returns the smallest list of digits that contains all of the digits of `a` in order, as well as all of the digits of `b` in order. A list is smaller than another if it is shorter, or if they have the same length and it comes first when compared digit by digit. You may use `smaller`, which returns the smaller of two lists.

```
pub fn smaller(s: List(Int), t: List(Int)) -> List(Int) {
  // Implementation omitted
}

pub fn combo(a: List(Int), b: List(Int)) -> List(Int) {
  case a, b {
    [], _ -> ____
    _, [] -> a
    [x, ..ar], [y, ..br] if x == y -> ____________________________
    [x, ..ar], [y, ..br] ->
      smaller(________________________, ________________________)
  }
}
assert combo([5, 3, 1], [4, 3, 2]) == [4, 5, 3, 1, 2]
assert combo([5, 3, 1], [4, 3, 2, 1]) == [4, 5, 3, 2, 1]
assert combo([1, 2, 3, 4], [9, 1, 2, 3]) == [9, 1, 2, 3, 4]
assert combo([], [3, 2, 1]) == [3, 2, 1]
```

5. *Adapted from CS 61A Fall 2023 Final Q6 (How to Get Promoted)*

(a) Implement `promote`, which takes a function `f` and a list `s`. It returns a list containing all of the elements of `s`, with the elements for which `f` returns `True` at the front. Otherwise, the elements stay in the same order as in `s`.

```
pub fn promote(f: fn(a) -> Bool, s: List(a)) -> List(a) {
  __________(list.filter(s, __________), list.filter(s, __________))
}
assert promote(fn(x) { x % 2 == 0 }, [1, 2, 3, 4, 5, 6, 7])
  == [2, 4, 6, 1, 3, 5, 7]
assert promote(fn(x) { x % 2 == 1 }, [1, 2, 3, 4, 5, 6, 7])
  == [1, 3, 5, 7, 2, 4, 6]
assert promote(fn(x) { x > 100 }, [1, 2, 3]) == [1, 2, 3]
```

(b) Implement `bigger_first`, which takes a non-empty list `s`. It returns a list with all of the elements of `s` that are larger than the first element at the front, followed by all of the elements that are smaller than or equal to the first element. Each group keeps its original order. You may use `promote` from the previous part and the functions below. **You may not write** `fn`, `case`, `[`, or `]` in the blank.

```
pub fn curry(f: fn(a, b) -> c) -> fn(a) -> fn(b) -> c {
  fn(x) { fn(y) { f(x, y) } }
}

pub fn curry_call(f: fn(x, y) -> r) -> fn(fn(z) -> x, fn(y) -> z) ->
    fn(y) -> r {
  fn(x, g) { fn(y) { f(x(g(y)), y) } }
}

pub fn less_than(x: Int, y: Int) -> Bool {
  x < y
}

pub fn first(s: List(Int)) -> Int {
  let assert [x, ..] = s
  x
}

pub fn bigger_first(s: List(Int)) -> List(Int) {
  ____________________________________________________(s)
}
assert bigger_first([3, 1, 4, 1, 5, 9, 2, 6]) == [4, 5, 9, 6, 3, 1, 1,
    2]
assert bigger_first([5, 1, 2]) == [5, 1, 2]
assert bigger_first([1, 2, 3]) == [2, 3, 1]
```

2 Interpreters

Interpreters can seem daunting to design, since an interpreter is a program that can carry out any possible computation depending on its input. Yet many share an elegant structure built from two mutually recursive functions: `eval`, which takes an expression and an environment and returns a value, and `apply`, which takes a function and a list of argument values and returns a value. An environment is a mapping from names to values, and the two functions are defined in terms of each other because applying a user-defined function means evaluating its body in a new environment that binds the parameters to the arguments, while evaluating an expression may involve applying one or more functions.

`eval` handles each kind of expression differently: numbers evaluate to themselves, names are looked up in the environment, and calls evaluate their operator and operands before handing the results to `apply`. Special forms such as `case` are the exception, since they do not evaluate every subexpression and only run the branch that matches. Recursion stops at the simple cases (numbers and name lookups) and at primitive operations like `+`, which `apply` computes directly without evaluating a body. This back-and-forth is the eval-apply loop, and it continues until the whole program has been reduced to a single value.

Gleam's approach to types and pattern matching suits this design well. If expressions are represented by a type with one variant per form, such as `N`, `Var`, and `Binary`, then `eval` is a `case` expression with one branch per variant, and the compiler can warn you if you forget a form. To see how all of this plays out, we can count the calls to `eval` and `apply` as a program runs.

Sometimes it helps to visualize how an interpreter works by counting the calls to eval and apply as we run our code. There are some rules to it. Essentially, every time we encounter a new block of code whose value we're not sure of, we need to evaluate it. In a basic function call, we need to evaluate the function, then evaluate the operands, and only then can we apply the function to its operands. Here's a more detailed example:

```
//Here's the code we're going to run:
fn f(s) {
    case s {
        [] -> [3]
        [first, ..rest] -> [first, ..f(rest)]
    }
}
f([1, 1 + 1])

//And here's the corresponding series of calls to Eval and Apply:
eval f([1, 1 + 1])
  eval f
  eval [1, 1 + 1]
    eval 1
    eval 1 + 1
      eval +
      eval 1
      eval 1
      apply + to 1, 1 → 2
  apply f to [1, 2]
    eval case s { [] -> [3] [first, ..rest] -> ... }
      eval s
    eval [first, ..f(rest)]
      eval first
      eval f(rest)
        eval f
        eval rest
        apply f to [2]
          eval case s { [] -> [3] [first, ..rest] -> ... }
            eval s
          eval [first, ..f(rest)]
            eval first
```

```
            eval f(rest)
              eval f
              eval rest
              apply f to []
                eval case s { [] -> [3] [first, ..rest] -> ... }
                  eval s
                eval [3]
                  eval 3
              apply f to [] → [3]
        apply f to [2] → [2, 3]
  apply f to [1, 2] → [1, 2, 3] //Here it repeats the earlier "apply f to [1,2]", this is not
      actually an additional apply call, it just restated this apply call once it found the
      result.
result: [1, 2, 3]
```

Something worth noting (also shown as a comment in this trace), is that when we have this gleam interpreter print out the eval/apply trace, if the apply call ends up taking several more steps before finding the result, it will re-print the earlier apply call immediately once it's found the result. We can see this with the call to "apply f to [1, 2]", it prints this and then once it's done applying, it prints "apply f to [1, 2] → [1, 2, 3]". We don't count this as an additional call to apply, it just printed this again to demonstrate that it found the result. Conversely, we can see that this call to apply "apply + to 1, 1 → 2" was only printed once, because it immediately found the result and didn't need to have any additional calls to eval/apply before finding the result. Ultimately, if we're looking at this type of trace, we have to make sure we disregard any repeated printed calls to apply in order to come to the correct number.

1. Identify the number of calls to **`eval`** and the number of calls to **`apply`** in `evaluator.gleam` made when `run` calls `draw` in each program below. (`run` evaluates the body of `draw` directly, so `draw` itself is not an **`apply`**.) For each program, count the calls made while evaluating that program, then choose one number from each row of the table. The numbers in the table are answer choices, not calls to add together.

   (a)
```
fn draw(shape_slider: Int) -> Int {
  case True {
    True -> abs(3)
    False -> abs(1)
  }
}
```

eval	4	5	7	9
apply	1	2	3	4

   (b)
```
fn draw(shape_slider: Int) -> Point {
  case abs(2) {
    2 -> Point(1, 2)
    _ -> Point(0, 0)
  }
}
```

eval	6	8	10	12
apply	1	2	3	4

   (c)
```
fn twin(x: Int) -> Point {
  Point(x, x)
}

fn draw(shape_slider: Int) -> Point {
  twin(abs(3))
}
```

eval	6	9	11	14
apply	1	2	3	4

(d)
```
fn pair(x: Int, y: Int) -> Point {
  Point(x, y)
}

fn draw(shape_slider: Int) -> Point {
  pair(abs(3), case True {
    True -> 2
    False -> 0
  })
}
```

eval	10	12	13	15
apply	1	2	3	4

The questions in this section use the arithmetic expression language that you saw from homework, but they do not require finishing the homework.

Expressions

An arithmetic expression is either a variable, an integer, or one of the operators `+`, `-`, `*`, or `/` applied to two smaller expressions. Each case is a variant of `Expr`:

```
pub type Operator {
  Add
  Sub
  Mul
  Div
}

pub type Expr {
  Var(String)
  N(Int)
  Binary(operator: Operator, left: Expr, right: Expr)
}
```

For example, `(x * (2 + 3))` is represented as

```
Binary(Mul, Var("x"), Binary(Add, N(2), N(3)))
```

Note that `Binary` lists its operator *before* its left and right expressions. A variable's name is a single letter from `a` to `z`. In this section, we often write an expression like `(x * (2 + 3))` as shorthand for its `Expr`, with every `Binary` expression in parentheses.

Two helper functions are provided in the homework:

- `symbol(operator: Operator)-> String` returns the symbol for an operator, such as `"+"` for `Add`.
- `calculate(operator: Operator, a: Int, b: Int)-> Int` applies an operator to two integers, so `calculate(Mul, 2, 3)` is `6`. As in Gleam, `/` drops any remainder, and dividing by zero gives `0`.

```
pub fn symbol(operator: Operator) -> String {
  case operator {
    Add -> "+"
    Sub -> "-"
    Mul -> "*"
    Div -> "/"
  }
}

pub fn calculate(operator: Operator, a: Int, b: Int) -> Int {
  case operator {
    Add -> a + b
    Sub -> a - b
    Mul -> a * b
    Div -> a / b
  }
}
```

Functions

A function such as `fn(x, y){ 2 * y - x }` has a list of parameters and a body expression, which a `UserFn` holds:

```
pub type UserFn {
  UserFn(params: List(String), body: Expr)
}
```

For example, `fn(x, y){ ((2 * y)- x)}` is `UserFn(["x", "y"], Binary(Sub, Binary(Mul, N(2), Var("y")), Var("x")))`. A function is *valid* if no parameter is repeated and every variable in its body is one of its parameters. You may assume every `UserFn` passed to a function in this section is valid.

Errors are represented by an `EvalError`. In this section, we only use `WrongArgumentCount`, which records how many arguments a function wants and how many it got:

```
pub type EvalError {
  NameNotFound(String)
  WrongArgumentCount(want: Int, got: Int)
}
```

Homework Functions

For reference, the homework asks you to implement the following functions. The questions in this section don't call them, but they are a good way to think about what each question is doing.

- `to_string(expr)` returns an expression written out as a string, with every `Binary` expression in parentheses. For example, the `Expr` above becomes `"(x * (2 + 3))"`.
- **`eval`**`(expr, bindings)` returns the value of an expression when each variable has the value it is bound to in the `Dict` `bindings`, or a `NameNotFound` error for a variable that isn't bound.
- `make_fn(params, body)` returns a `UserFn` if the function is valid, or an error listing the repeated parameters or the variables in the body that aren't parameters.
- **`apply`**`(f, args)` returns the value of the body of `f` when each parameter is bound to the argument in the same position, or a `WrongArgumentCount` error if the number of arguments is wrong. For example, applying `fn(x, y){ ((2 * y)- x)}` to `1` and `5` gives `9`.

2. Implement `simplify`, which takes in an `Expr expr` and returns an equivalent expression in which every `Binary` expression whose two sides both simplify to numbers is replaced by the single number it computes. Parts of `expr` that contain variables are kept. For example, `(x + (2 * 3))` simplifies to `(x + 6)`. Use the provided `calculate` function.

```
pub fn simplify(expr: Expr) -> Expr {
  case expr {
    Binary(operator, left, right) ->
      case _____________(__________), _____________(__________) {
        N(a), N(b) -> ________________________________________
        l, r -> ______________________________________
      }
    _ -> _____________
  }
}

assert simplify(Binary(Add, Var("x"), Binary(Mul, N(2), N(3))))
  == Binary(Add, Var("x"), N(6))
assert simplify(Binary(Mul, Binary(Sub, N(5), N(1)), N(2))) == N(8)
assert simplify(Binary(Add, Binary(Add, Var("x"), N(1)), Binary(Add, N(2),
   N(3))))
  == Binary(Add, Binary(Add, Var("x"), N(1)), N(5))
assert simplify(Var("x")) == Var("x")
```

3. Implement `substitute`, which takes in an `Expr expr`, a variable `name`, and an `Expr replacement`, and returns `expr` with every occurrence of the variable `name` replaced by `replacement`. All other parts of `expr` stay the same. For example, substituting `N(4)` for `"x"` in `(x + y)` gives `(4 + y)`.

```
pub fn substitute(expr: Expr, name: String, replacement: Expr) -> Expr {
  case expr {
    Var(v) if _____________ -> _____________
    Var(_) | N(_) -> _____________
    Binary(operator, left, right) -> {
      let new_left = _____________(_____________, _____________, _____________)
      let new_right = _____________(_____________, _____________, _____________)
      Binary(_____________, _____________, _____________)
    }
  }
}

assert substitute(Binary(Add, Var("x"), Var("y")), "x", N(4))
  == Binary(Add, N(4), Var("y"))
assert substitute(Binary(Mul, Var("x"), Var("x")), "x", Binary(Sub,
   Var("a"), N(1)))
  == Binary(Mul, Binary(Sub, Var("a"), N(1)), Binary(Sub, Var("a"), N(1)))
assert substitute(Binary(Mul, N(2), Var("y")), "x", N(9))
  == Binary(Mul, N(2), Var("y"))
```

4. Implement `curry`, which takes in a `UserFn f` and an integer `arg`, and returns a function that takes all but the first parameter of `f`, with `arg` bound to the first parameter. For example, currying `fn(x, y){ (y - x)}` with `1` gives `fn(y){ (y - 1)}`. If `f` has no parameters, return `Error(WrongArgumentCount(want: 0, got: 1))`. You may use `substitute` from the previous question.

```
pub fn curry(f: UserFn, arg: Int) -> Result(UserFn, EvalError) {
  case f.params {
    [] -> Error(WrongArgumentCount(want: 0, got: 1))
    [first, ..rest] -> {
      let new_body = _____________(____________, ____________, ____________)
      Ok(UserFn(____________, ____________))
    }
  }
}

assert curry(UserFn(["x", "y"], Binary(Sub, Var("y"), Var("x"))), 1)
  == Ok(UserFn(["y"], Binary(Sub, Var("y"), N(1))))
assert curry(UserFn(["c"], Binary(Add, N(10), Var("c"))), 1)
  == Ok(UserFn([], Binary(Add, N(10), N(1))))
assert curry(UserFn([], N(7)), 1) == Error(WrongArgumentCount(want: 0,
    got: 1))
```

5. Implement `compose`, which takes in two `UserFn`s `f` and `g`, and returns a function `h` such that applying `h` to some arguments gives the same result as applying `g` to those arguments and then applying `f` to that result. For example, composing `fn(x){ (x * x)}` with `fn(x, y){ (y - x)}` gives `fn(x, y){ ((y - x)* (y - x))}`. Since `f` receives exactly one value, return a `WrongArgumentCount` error if `f` does not take exactly one parameter. You may use `substitute`.

```
pub fn compose(f: UserFn, g: UserFn) -> Result(UserFn, EvalError) {
  case f.params {
    _______ -> {
      let new_body = _____________(____________, ____________, ____________)
      Ok(UserFn(____________, ____________))
    }
    params -> Error(WrongArgumentCount(want: ____________(_______), got: 1))
  }
}

assert compose(
    UserFn(["x"], Binary(Mul, Var("x"), Var("x"))),
    UserFn(["x", "y"], Binary(Sub, Var("y"), Var("x"))),
  )
  == Ok(UserFn(
    ["x", "y"],
    Binary(Mul, Binary(Sub, Var("y"), Var("x")), Binary(Sub, Var("y"),
       Var("x"))),
  ))
assert compose(UserFn(["x", "y"], Binary(Sub, Var("y"), Var("x"))),
    UserFn(["x"], Var("x")))
  == Error(WrongArgumentCount(want: 2, got: 1))
```